

Integración de Sistemas IoT con ThingSpeak

Memoria técnica de la aplicación desarrollada para crear, consultar, visualizar y gestionar datos de un canal IoT mediante la API REST de ThingSpeak.

Fecha de elaboración

29 de abril de 2026

Canal creado para la tarea

Estación meteorológica David Orive

Aplicación entregada

Panel web en Python con Flask y cliente HTTP para la API de ThingSpeak

Ficheros principales

app.py, thingspeak_client.py, templates/index.html y static/styles.css

Índice

- 1. Objetivo de la práctica
- 2. Creación del canal y configuración
- 3. Lectura y escritura mediante API
- 4. Aplicación desarrollada
- 5. Fragmentos principales del código
- 6. Conclusiones

1. Objetivo de la práctica

El objetivo de la práctica es utilizar la plataforma ThingSpeak para gestionar los datos de un sistema IoT. Para ello se ha creado un canal propio en la cuenta del alumno, se han definido ocho campos relacionados con una estación meteorológica y se ha desarrollado una aplicación web en Python que consume la API REST del servicio para leer feeds, consultar estados, representar gráficas y permitir el envío de nuevos datos al canal.

El flujo completo de la práctica ha quedado cubierto: creación real del canal, obtención de las claves de acceso, inserción de lecturas de prueba mediante API y comprobación de la aplicación local conectada al canal creado específicamente para esta tarea.

2. Creación del canal y configuración

Elemento	Valor configurado
Nombre del canal	Estación meteorológica David Orive
Channel ID	3363654
Author	mwa0000041264990
Access	Private
Field 1	Temperatura
Field 2	Humedad
Field 3	Lluvia
Field 4	Calidad del aire
Field 5	Estado

Elemento	Valor configurado
Field 6	Nivel de luz
Field 7	GPS
Field 8	Presión

Private ViewPublic ViewChannel SettingsSharingAPI KeysData Import / Export

Write API Key

Key

Generate New Write API Key

Read API Keys

Key

Note

Save NoteDelete API Key

Add New Read API Key

Help

API keys enable you to write data to a channel or read data from a private channel. API keys are auto-generated when you create a new channel.

API Keys Settings

Write API Key: Use this key to write data to a channel. If you feel your key has been compromised, click **Generate New Write API Key**.

Read API Keys: Use this key to allow other people to view your private channel feeds and charts. Click **Generate New Read API Key** to generate an additional read key for the channel.

Note: Use this field to enter information about channel read keys. For example, add notes to keep track of users with access to your channel.

API Requests

Write a Channel Feed

GET

Read a Channel Feed

GET https://api.thingspeak.com/channels/3363654/feeds.json?api_key=[READ_API_KEY]&results=15

Read a Channel Field

GET https://api.thingspeak.com/channels/3363654/fields/1.json?api_key=[READ_API_KEY]&results=15

Figura 1. Pestaña API Keys del canal creado en ThingSpeak. En la memoria se han ocultado los valores sensibles de las claves para no exponer credenciales del canal.

3. Lectura y escritura mediante API

Una vez creado el canal, la gestión de los datos se ha realizado a través de la API REST de ThingSpeak. La escritura se probó con la Write API Key real del canal, aunque por seguridad no se incluye dicha clave en esta memoria. Sí se muestran las rutas de lectura requeridas por el enunciado, ya que son las que utiliza la aplicación para consultar los datos del canal privado.

Operación	Dirección utilizada
Read a channel feed	https://api.thingspeak.com/channels/3363654/feeds.json?api_key=[READ_API_KEY]&results=15
Read a channel field	https://api.thingspeak.com/channels/3363654/fields/1.json?api_key=[READ_API_KEY]&results=15
Read channel status updates	https://api.thingspeak.com/channels/3363654/status.json?api_key=[READ_API_KEY]

Para verificar el funcionamiento de la escritura, se enviaron tres lecturas de ejemplo al canal. Los datos recibidos por ThingSpeak fueron los siguientes:

Entry ID	Temperatura	Humedad	Lluvia	Calidad aire	Estado	Nivel luz	GPS	Presión	Status
1	24.8	58.1	0	42	1	560	28.466,-16.255	1013	Lectura inicial desde aplicación
2	24.4	59.0	1	47	1	590	28.466,-16.255	1011	Muestreo 3
3	25.1	57.4	0	40	1	540	28.466,-16.255	1012	Muestreo 2

4. Aplicación desarrollada

La solución implementada consiste en una aplicación web ligera en Python con Flask. La aplicación permite:

- Cargar un canal indicando Channel ID, Read API Key y número de registros.
- Construir automáticamente las direcciones de lectura del canal.
- Consultar los feeds y los *status updates* recientes.
- Representar una gráfica por cada campo disponible.
- Preparar el envío de nuevos datos al canal cuando se facilita la Write API Key.

Panel de gestión para canales de ThingSpeak

La aplicación consulta un canal, muestra sus feeds y estados recientes, genera gráficas y permite publicar nuevos valores cuando se dispone de la Write API Key.

Configuración del canal

Channel ID

3363654

Read API Key

[READ_API_KEY]

Write API Key

Necesaria para enviar datos

Registros

15

Cargar canal

URLs generadas por la app

Read a channel feed:

[https://api.thingspeak.com/channels/3363654/feeds.json?results=15&api_key=\[READ_API_KEY\]](https://api.thingspeak.com/channels/3363654/feeds.json?results=15&api_key=[READ_API_KEY])

Read channel status updates:

[https://api.thingspeak.com/channels/3363654/status.json?results=15&api_key=\[READ_API_KEY\]](https://api.thingspeak.com/channels/3363654/status.json?results=15&api_key=[READ_API_KEY])

Enviar nuevos datos

Field 1

Valor opcional

Field 2

Valor opcional

Field 3

Valor opcional

Field 4

Valor opcional

Field 5

Valor opcional

Field 6

Valor opcional

Field 7

Valor opcional

Field 8

Valor opcional

Status

Comentario opcional para la actualización

Enviar actualización

El proyecto queda configurado por defecto con el canal de la tarea. Por seguridad, la Write API Key no se guarda en los ficheros y se introduce solo cuando hace falta enviar nuevos datos.

Canal

**Estación meteorológica
David Orive**
ID 3363654

Última entrada

3
Feeds recuperados: 3

Ubicación

0.0, 0.0
Elevación: No indicada

Estados

3
Actualizaciones consultadas

Metadatos del canal

Figura 2. Vista superior de la aplicación local conectada al canal privado. La captura pública sustituye la Read API Key por un marcador para no exponer credenciales.

Metadatos del canal

Nombre

Estación meteorológica David Orive

Descripción

Canal para la gestión de datos ambientales de una estación IoT.

Creado

2026-04-29T17:48:07Z

Actualizado

2026-04-29T17:48:08Z

field1

Temperatura

field2

Humedad

field3

Lluvia

field4

Calidad del aire

field5

Estado

field6

Nivel de luz

field7

GPS

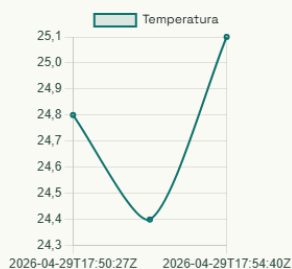
field8

Presión

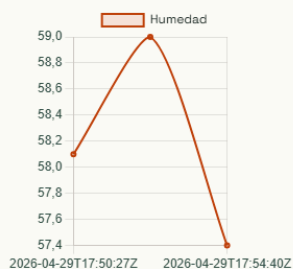
Figura 3. Sección de metadatos del canal con nombre, descripción, fechas y etiquetas de los ocho campos recuperados desde ThingSpeak.

Gráficas del canal

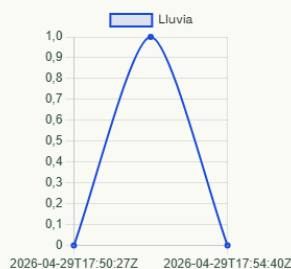
Temperatura



Humedad



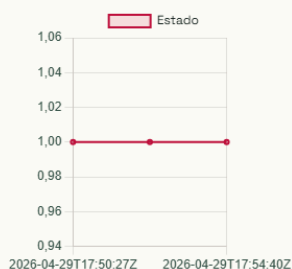
Lluvia



Calidad del aire



Estado



Nivel de luz



GPS



Presión

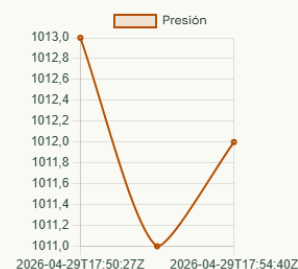


Figura 4. Gráficas generadas por la aplicación para los ocho campos del canal, construidas a partir de los feeds almacenados en ThingSpeak.

Feeds recientes

Fecha	Entry ID	Temperatura	Humedad	Lluvia	Calidad del aire	Estado	Nivel de luz	GPS	Presión
2026-04-29T17:50:27Z	1	24,8	58,1	0	42	1	560	28.466,-16.255	1013
2026-04-29T17:54:02Z	2	24,4	59,0	1	47	1	590	28.466,-16.255	1011
2026-04-29T17:54:40Z	3	25,1	57,4	0	40	1	540	28.466,-16.255	1012

Status updates

Fecha	Entry ID	Status
2026-04-29T17:50:27Z	1	Lectura inicial desde aplicacion
2026-04-29T17:54:02Z	2	Muestreo 3
2026-04-29T17:54:40Z	3	Muestreo 2

Figura 5. Tablas de feeds recientes y de status updates. La captura se ha preparado en una vista ampliada para que aparezcan completos todos los campos del registro.

Nota: la aplicación no guarda la Write API Key en los ficheros del proyecto. Para escribir nuevos datos puede introducirse manualmente en la interfaz o mediante una variable de entorno.

5. Fragmentos principales del código

5.1 Cliente de acceso a ThingSpeak

```
class ThingSpeakClient:
    def build_feed_url(self, config: ChannelConfig) -> str:
        params = [f"results={config.results}"]
        if config.read_api_key:
            params.append(f"api_key={config.read_api_key}")
        return f"{BASE_API_URL}/channels/{config.channel_id}/feeds.json?{'&'.join(params)}"

    def build_status_url(self, config: ChannelConfig) -> str:
        params = [f"results={config.results}"]
        if config.read_api_key:
```

```

        params.append(f"api_key={config.read_api_key}")
        return f"{BASE_API_URL}/channels/{config.channel_id}/status.json?{'&'.join(params)}"

def write_update(self, config: ChannelConfig, payload: dict[str, str]) -> str:
    update_url = self.build_update_url(config, payload)
    response = requests.get(update_url, timeout=self.timeout)
    response.raise_for_status()
    return response.text.strip()

```

5.2 Lógica principal de la aplicación

```

@APP.post("/send")
def send_data() -> Any:
    config = build_config_from_request()
    payload = {}
    for index in range(1, 9):
        value = request.form.get(f"field{index}", "").strip()
        if value:
            payload[f"field{index}"] = value

    status = request.form.get("status", "").strip()
    if status:
        payload["status"] = status

    entry_id = CLIENT.write_update(config, payload)
    flash(f"Actualización enviada correctamente. ThingSpeak devolvió entry id {entry_id}.", "success")

```

5.3 Preparación de series para las gráficas

```

def chart_payload(feeds: list[dict[str, Any]], labels: list[dict[str, str]]) -> list[dict[str, Any]]:
    charts = []
    timestamps = [feed["created_at"] for feed in feeds]
    for label in labels:
        series = []
        for feed in feeds:
            value = feed.get(label["key"])
            try:
                series.append(float(value) if value is not None else None)
            except (TypeError, ValueError):
                series.append(None)
        charts.append({"field": label["key"], "label": label["label"], "labels": timestamps, "data": series})
    return charts

```

Además de estos fragmentos, se entregan en el proyecto los ficheros completos `app.py`, `thingspeak_client.py`, `templates/index.html`, `static/styles.css` y las pruebas unitarias de `tests/test_thingspeak_client.py`.

6. Conclusiones

La práctica ha quedado completada de forma íntegra sobre un canal real de ThingSpeak. Se ha creado y configurado el canal, se han definido sus campos, se han obtenido las direcciones de lectura, se han insertado lecturas de prueba y se ha comprobado una aplicación propia capaz de gestionar sus datos. De esta forma se demuestra tanto el uso de la plataforma ThingSpeak como la integración de una aplicación externa que automatiza la consulta y visualización de la información del sistema IoT.